

COLA DE PROVA

Mobile Application Development

2TDSPV · FIAP · Prof. Felipe Cavalcante

React Native · Expo · TanStack Query · JWT · Firebase Auth · Cloud Firestore

Cada tópico tem: **O QUE FAZ** → **COMO IMPLEMENTAR** → **ONDE IMPLEMENTAR** → **PEGADINHAS**.
No final: tabelas de "não confunda", projeto-modelo completo e banco de perguntas com respostas.

Índice

PARTE A — Fundamentos (1º sem.)

- A1. Mundo mobile: nativo × cross × híbrido
- A2. Ambiente (Node, npm, npx, Expo Go)
- A3. React Native e Expo
- A4. Componentes, JSX, props, children, pureza
- A5. Estilos, Flexbox (Yoga), Platform, SafeArea
- A6. Design: HIG × Material 3, acessibilidade

PARTE B — TanStack Query

- B1. Aula 01 – useQuery e o ciclo da consulta
- B2. Aula 02 – cache, fresh/stale, staleTime, gcTime
- B3. Aula 03 – useMutation
- B4. Aula 04 – retry, idempotência, invalidação
- B5. Aula 05 – serviços + custom hooks

PARTE C — Identidade

- C1. Aula 06 – autenticação, sessão, JWT
- C2. Aula 07 – Firebase Auth e Stack.Protected

PARTE D — Cloud Firestore (BaaS)

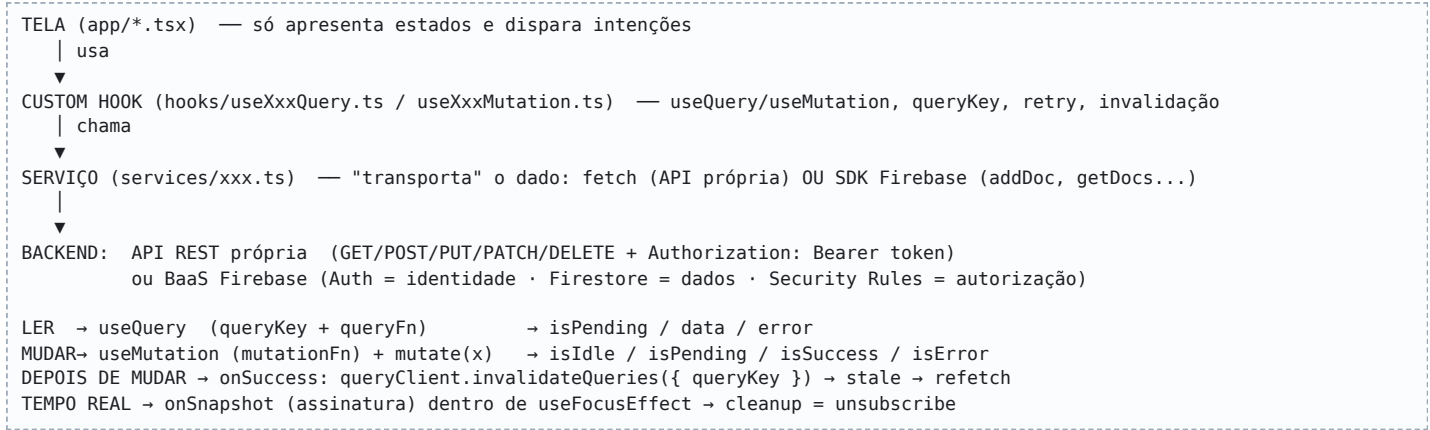
- D1. Aula 08 – BaaS, documentos, getDocs, Zod
- D2. Aula 09 – addDoc, serverTimestamp, Security Rules
- D3. Aula 10 – onSnapshot e useFocusEffect
- D4. Aula 11 – updateDoc e deleteDoc

PARTE E — Tabelas "não confunda"

PARTE F — Projeto-modelo (estrutura + arquivos)

PARTE G — Banco de perguntas e respostas

Mapa mental de tudo (decore este fluxo)



PARTE A — Fundamentos (1º semestre)

A1. O mundo mobile

PARA CRIAR UM APP VOCÊ PRECISA DE

SDK (Software Development Kit = kit com APIs da plataforma, runtime, build tools como Gradle/Xcode Build, emuladores e debuggers como ADB/LLDB) + **IDE** (Android Studio, Xcode – só macOS, VS Code para cross-platform) + **conhecimento de arquitetura mobile** (ciclo de vida do app, recursos limitados: bateria/memória/CPU, threads e operações assíncronas, persistência local: SQLite, Realm, AsyncStorage).

Tipo	Como é	Prós	Contras	Stacks / exemplos
Nativo	1 app por plataforma	Melhor performance, acesso total ao hardware, UX nativa	2× trabalho e 2× custo (Swift + Kotlin)	Swift/Kotlin · Instagram, Nubank
Cross-platform	1 código p/ iOS e Android (70–90% compartilhado)	Time único, –40 a 60% tempo	Performance próxima (não igual), limitação em recursos muito específicos	React Native, Flutter, Kotlin Multiplatform · Discord, Shopify
Híbrido	App web dentro de WebView	Reusa código web, rápido	Performance inferior, "cara de web"	Ionic, Cordova, Capacitor · MVPs

Quando usar: games/AR → Nativo · startup com pouco budget, app de conteúdo/e-commerce, time pequeno → Cross-platform · MVP rápido → Híbrido.

Aspecto	iOS	Android
Linguagens / IDE	Swift, Obj-C / Xcode (macOS)	Kotlin, Java / Android Studio
Market share	~30% global, ~18% BR	~70% global, ~82% BR
Loja / custo	App Store, revisão rigorosa (1–3 dias), US\$ 99/ano	Google Play, horas (automatizada), US\$ 25 único
Fragmentação	Baixa	Alta
Design system	HIG (Human Interface Guidelines) – clareza, deferência, profundidade · toque mín. 44pt · fonte SF Pro	Material Design – superfícies, FAB, Bottom Nav · toque mín. 48dp · fonte Roboto
Voltar / modal	Swipe back / swipe down	Botão voltar

A2. Ambiente de desenvolvimento

- Node.js** = runtime JavaScript (roda JS fora do navegador). Use sempre a versão **LTS** (Long Term Support – estável, com correções de segurança). *Current* = novidades, pode ser instável.
- npm** = gerenciador de pacotes (vem com o Node). **npx** = executa um pacote sem instalar globalmente.
- Validar: `node -v`, `npm -v`, `npx -v`.
- Expo Go** = app no celular que roda seu projeto lendo o QR Code. Opcionais: JDK, Android Studio (emulador), Xcode.

```
npx create-expo-app@latest hello-world      # cria o projeto
cd hello-world
npx expo start                             # sobe o servidor (Metro) e mostra o QR Code
```

A3. React Native e Expo

O QUE É

React Native: framework cross-platform. Você escreve em TypeScript (transpilado p/ JS), o código é empacotado (bundle) e enviado ao celular; a **JSI** (JavaScript Interface) faz o JS conversar com o código nativo. Os componentes viram componentes **nativos de verdade** (não é WebView); a lógica continua em JS.

Metro = bundler (empacotador) do RN. **Fast Refresh** = atualiza o app ao salvar, sem rebuild, mantendo o estado quando possível.

Expo = framework sobre o RN: começa rápido, sem configurar ambiente nativo, SDK rico (câmera, notificações, sensores), build/deploy simplificados.

```
npx @react-native-community/cli@latest init FirstApp  # RN "puro" (precisa Android Studio/Xcode)
npm start                                           # inicia o Metro
npm run android

npx create-expo-app@latest app-starter --template    # Expo: escolha Blank (TypeScript)
```

Arquivo (Expo)	Papel
app.json	configuração principal do app (nome, versão, ícone...)
App.tsx / app/_layout.tsx (Expo Router)	componente raiz / layout raiz das rotas
index.ts	ponto de entrada
assets/	imagens, fontes
package.json / package-lock.json	dependências e scripts / versões travadas
tsconfig.json	config do TypeScript
.expo/	cache interno — não editar

RN CLI tem também `android/` e `ios/` (código nativo), `metro.config.js`, `babel.config.js` (transpiler), `Gemfile` (Ruby p/ iOS).

A4. Componentes, JSX e props (React Architecture)

CONCEITOS

Componente = função JS que retorna interface (`export default function Profile() {...}`). **JSX** = sintaxe parecida com HTML dentro do JS. Regras: **uma única raiz** (use `<>...</>` = Fragment), **toda tag fechada** (`<Image />`), atributos em **camelCase** (`className` na web).

Árvore de renderização: componentes de topo (raiz) passam dados para baixo; componentes folha só retornam marcação.

Props = "botões de ajuste" que o pai passa ao filho. São **imutáveis** — o filho nunca altera props.

children = prop especial com o conteúdo colocado "dentro" do componente (Cards, Layouts).

Pureza: `UI = f(props)` — mesmas entradas → mesma saída. Alterar variável externa/global quebra a pureza. **Mutação local** (de algo criado durante a própria renderização) é OK.

Cascata: se o pai re-renderiza, os filhos re-renderizam por padrão. Passar componente pesado via `children` evita re-render (otimização por composição).

```
function Avatar({ person, size = 100 }) { ... } // destructuring + valor padrão
<Card><Avatar person={user} /></Card>           // children

if (isPacked) return null;                      // não renderizar nada
{isPacked ? <Check /> : <Cross />}              // ternário
{isPacked && <Check />}                          // AND lógico
{count > 0 && <Badge />} // NUNCA {count && ...}: se count=0 o React renderiza "0" (no RN pode até quebrar)
```

Web (DOM)	React Native	Obs.
<code><div></code>	<code><View></code>	bloco fundamental
<code>/<p></code>	<code><Text></code>	texto solto fora de <Text> quebra o app
<code><button></code>	<code><Pressable></code> , <code><TouchableOpacity></code> , <code><Button></code>	evento é <code>onPress</code> (não <code>onClick</code>)
<code></code>	<code><Image></code>	
<code>ul/map</code>	<code><FlatList data renderItem keyExtractor></code>	lista virtualizada (performática)
spinner	<code><ActivityIndicator /></code>	loading

A5. Estilos e layout

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center" },
  text: { color: "blue", fontSize: 20, marginTop: 10 }, // camelCase, números sem "px"
});
<View style={[styles.base, isActive && styles.active]} /> // array = estilos dinâmicos
```

- Sem CSS/cascata**: estilos são objetos JS, encapsulados no componente. **camelCase** (`marginTop`).
- Yoga** = motor de layout que implementa Flexbox. **Diferença crítica: flexDirection padrão no RN é column** (na web é row).
- flex**: número divide o espaço: filhos com flex 1, 2, 3 → espaço dividido em 6 partes.
- justifyContent** = eixo **principal**; **alignItems** = eixo **cruzado**. Centralizar em 2D: os dois = `'center'`.
- flexWrap**: `'wrap'` + **flexDirection**: `'row'` para chips/tags (`wrap-reverse` não existe no RN).
- gap** pode ser inconsistente entre versões → usar `marginBottom` ou componente `<Spacer size={8} />`.
- Sombras: iOS = `shadowColor`, `shadowOffset`, `shadowOpacity`, `shadowRadius`; Android = `elevation`. Unificar com `Platform.select({ ios: {...}, android: {...} })`.
- Responsividade sem media queries: `const { width } = useWindowDimensions(); → width > 600 ? <Tablet/> : <Celular/>`. Unidade = dp (pixels independentes de densidade).
- `<SafeAreaView>` adiciona padding para não ficar atrás do notch/câmera/barra.

A6. Design mobile (HIG × Material 3)

Tema	Apple (HIG / Liquid Glass)	Google (Material You / MD3)
Filosofia	Física óptica, vidro translúcido, profundidade por materiais	Emoção e personalização; Dynamic Color extrai cores do wallpaper
Cores	Cores semânticas (<code>systemRed</code> , <code>labelColor</code>) que se adaptam	Tokens (<code>colorPrimaryContainer</code> , <code>onSurface</code>) — nunca hex fixo
Dark mode	Cores semânticas; camadas Base × Elevated (sem sombras)	Sobreposições tonais
Botões	Cápsulas/círculos, estilo <code>.prominent</code> ; não customizar texto "Voltar"	FAB; hierarquia Filled > Tonal > Outlined
Navegação	Celular: Navigation Bar/Tab Bar · Tablet: Navigation Rail/Sidebar	
Toque mínimo	44pt (visionOS 60pt+)	48dp ; espaçamento mín. ~12pt entre controles
Motion	Feedback: visual + áudio + haptics; spinner imediato em ação demorada	Springs (Expressive p/ destaque, Standard p/ utilitário)

- **Window Size Classes:** Compact (phone), Medium (foldable/tablet), Expanded (desktop) → o *conteúdo* dita o layout (List/Detail lado a lado em telas grandes).
- **Acessibilidade:** labels em ícones, contraste $\geq 4.5:1$ p/ texto pequeno, botões semânticos (não imagens clicáveis), respeitar "Reduce Motion", **ações destrutivas exigem confirmação**, nunca rasterizar texto em imagem.
- Widgets: Apple tamanhos fixos (Smart Stack/StandBy); Android redimensionáveis em grade de 48dp.
- 3 regras de ouro: **pense adaptativo · respeite a plataforma · acessibilidade primeiro.**

PARTE B — TanStack Query (React Query)

B1. Aula 01 — Buscar estado: `useQuery`

PROBLEMA E O QUE FAZ

Buscar dados remotos não é só guardar *dados*: é gerenciar o **estado da consulta** (aguardando, sucesso, erro). Manualmente isso exige `useState (data, isLoading, error) + useEffect + try/catch/finally` em toda tela.

TanStack Query é uma biblioteca que **organiza o estado produzido pelas consultas remotas**. Ele **NÃO substitui o `fetch`** — você continua escrevendo a função que busca; ele acompanha o ciclo (`pending` → `data/error`), faz cache, `refetch` e `retry`.

Query = consulta de **leitura**. Responde duas perguntas: **qual consulta?** → `queryKey` · **como buscar?** → `queryFn`.

Passo 1 — instalar

```
npm i @tanstack/react-query
```

Passo 2 — configurar o provider (uma vez, na raiz)

```
app/_layout.tsx

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { Stack } from "expo-router";

const queryClient = new QueryClient(); // FORA do componente (senão recria o cache a cada render)

export default function Layout() {
  return (
    <QueryClientProvider client={queryClient}> /* disponibiliza as queries p/ todo o app */
      <Stack />
    </QueryClientProvider>
  );
}
```

Passo 3 — a função que busca (`queryFn`)

```
services/movies.ts

export async function fetchMovies() {
  const response = await fetch(`${API_URL}/movies`);
  if (!response.ok) throw new Error("Falha ao carregar"); // fetch NÃO lança erro em 404/500 → você precisa lançar
  return response.json();
}
```

Passo 4 — usar na tela

```
app/index.tsx

const { data, isPending, error } = useQuery({
  queryKey: ["movies"], // identifica a consulta (e os dados no cache)
  queryFn: fetchMovies, // passa a REFERÊNCIA da função (sem parênteses)
});

if (isPending) return <ActivityIndicator />;
if (error) return <Text>Erro ao carregar: {error.message}</Text>;
if (data.length === 0) return <Text>Nenhum tweet encontrado.</Text>; // desafio: lista vazia

return (
  <FlatList
    data={data}
    renderItem={({ item }) => <Text>{item.title}</Text>}
    keyExtractor={(item) => item.id.toString()}
  />
);
```

Situação	Propriedade do <code>useQuery</code>
Aguardando resposta (ainda não há dado)	<code>isPending</code>
Dados recebidos	<code>data (+ isSuccess)</code>
Falha	<code>error (+ isError)</code>
Uma busca em andamento (inclusive <code>refetch</code> em 2º plano)	<code>isFetching</code>
Buscar de novo manualmente	<code>refetch()</code>

PEGADINHAS

- `queryFn: fetchMovies()` (com parênteses) está ERRADO — executaria na hora e passaria uma `Promise`.
- Não esqueça o `QueryClientProvider`: sem ele, `useQuery` dá erro "No QueryClient set".
- Ordem dos ifs: `pending` → `error` → `vazio` → `lista`. • A responsabilidade do ciclo "deixa de ficar espalhada pelo componente".

B2. Aula 02 — Mantendo dados remotos atualizados (cache)

CONCEITOS (CAI MUITO!)

Cache: onde o TanStack Query guarda os dados obtidos. Ter dado no cache ≠ dado atual.

queryKey: identifica os dados no cache. **Mesma chave = mesmos dados compartilhados** entre componentes.

fresh: dado considerado atual → pode ser reutilizado sem buscar.

stale: dado continua no cache, mas *pode* estar desatualizado → candidato a refetch.

staleTime: quanto tempo o dado fica **fresh**. **Padrão = 0** (fica stale imediatamente — por isso ao voltar à tela há nova requisição).

refetch: buscar de novo um dado já carregado. Acontece em **oportunidades:** componente monta/passa a usar a query, app recupera o foco, rede reconecta.

active: ≥ 1 componente usando a query. **inactive:** nenhum usa (dados continuam no cache).

gcTime (garbage collection time): quanto tempo uma query **inativa** fica no cache antes de ser removida (padrão 5 min). Depois disso, nova consulta "do zero".

```
useQuery({
  queryKey: ["products"],
  queryFn: getProducts,
  staleTime: 30_000, // fresh por 30 s → voltar à tela dentro de 30 s NÃO refaz requisição
  gcTime: 10_000,    // inativa por mais de 10 s → removida do cache
});
```

Dois ciclos independentes:

fresh —(staleTime)→ stale → "quanto confio na atualidade do dado"

active —(sem consumidores)→ inactive —(gcTime)→ garbage collected
→ "está em uso? por quanto tempo guardo sem uso?"

Voltando à tela: dado no cache ✓ → está stale ✓ → componente voltou a usar ✓ → REFETCH (dado antigo aparece na hora, e é substituído quando o novo chega)

Devtools no Expo

```
npx expo install @dev-plugins/react-query
import { useReactQueryDevTools } from "@dev-plugins/react-query";
useReactQueryDevTools(queryClient); // no componente raiz; no terminal: shift + m → React Query
```

TRADE-OFF (PERGUNTA DISSERTATIVA)

Buscar sempre = dados mais atuais, mais requisições. Reutilizar sempre = menos requisições, risco de dado velho. staleTime é o "botão" que equilibra isso por consulta (ex.: cotação → staleTime baixo; lista de países → alto).

B3. Aula 03 — Alterando dados remotos: `useMutation`

O QUE FAZ

Query = obter dados (GET; o servidor é a fonte). **Mutation** = **causar uma mudança** (POST, PUT, PATCH, DELETE). Ex.: criar filme, alterar pedido, excluir comentário, confirmar reserva, publicar mensagem.

Cada ação = uma mutation diferente (criar, editar, excluir têm mutationFn e ciclo próprios).

mutationFn = o que executar (função assíncrona que fala com a API). **mutate()** = quando executar (normalmente após ação do usuário).

Diferença-chave: a **query executa sozinha** (queremos observar dados); a **mutation espera alguém chamar mutate(...)**. Criar a mutation não faz nada.

services/tweets.ts

```
type NewTweet = { text: string };

export async function createTweet(tweet: NewTweet) {
  const response = await fetch(`${API_URL}/tweets`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(tweet),
  });
  if (!response.ok) throw new Error("Não foi possível adicionar o tweet");
  return response.json();
}

export async function deleteTweet(id: number) { // desafio da aula
  const response = await fetch(`${API_URL}/tweets/${id}`, { method: "DELETE" });
  if (!response.ok) throw new Error("Não foi possível excluir");
}
```

app/index.tsx (antes de refatorar p/ hook)

```
const [text, setText] = useState("");
const { mutate, isPending, isError, isSuccess, error } = useMutation({
  mutationFn: createTweet,
  onSuccess: () => setText(""), // consequência do sucesso
  onError: () => Alert.alert("Erro", "Não foi possível tweetar."), // consequência do erro
  onSettled: () => { /* roda no fim, com sucesso OU erro */ },
});

<Pressable disabled={isPending} onPress={() => mutate({ text })}>
  <Text>{isPending ? "Salvando..." : "Tweetar"}</Text>
</Pressable>
{isError && <Text>Não foi possível tweetar: {error.message}</Text>}
{isSuccess && <Text>Tweet adicionado com sucesso!</Text>}

// Excluir: mutate recebe o id
const deleteMutation = useMutation({ mutationFn: deleteTweet });
<Pressable disabled={deleteMutation.isPending} onPress={() => deleteMutation.mutate(item.id)}>
```

Estado da mutation (um por vez)	Significado
isIdle	ainda não começou
isPending	esta operação está sendo executada (não é "o app está carregando")
isSuccess	terminou com sucesso
isError	terminou com erro (error.message)

```
mutate(tweet) → mutationFn = createTweet(tweet) → POST /tweets
|
mutationFn EXECUTA a operação · callbacks (onSuccess/onError/onSettled) REAGEM ao resultado
```

PEGADINHAS

- Bloquear cliques duplos com `disabled={isPending}`.
- `mutationFn` recebe **um único argumento** (se precisar de vários, passe um objeto).
- A mutation sozinha **não atualiza a lista** — isso é a aula 04 (invalidação).
- Callbacks também podem ir no próprio `mutate(x, { onSuccess })` (usado no login).

B4. Aula 04 — Mantendo a interface consistente

Retry (quando falha)

O QUE É

Retry = nova tentativa da **mesma operação** depois de uma falha. Nem toda falha é definitiva (rede oscilou, servidor fora por segundos, timeout).

Padrão: Query tenta de novo até 3 vezes; Mutation NÃO tenta de novo — porque repetir uma mudança pode produzir novos efeitos (duplicar).

```
useQuery({ queryKey: ["tweets"], queryFn: fetchTweets, retry: 2 }); // 1 tentativa inicial + 2 novas = 3 no total
useMutation({ mutationFn: createTweet, retry: false }); // explícito: sem retry
```

Retry PODE ajudar	Retry NÃO resolve
conexão instável · timeout · servidor temporariamente indisponível	endereço incorreto (404) · dados inválidos (400) · acesso não autorizado (401/403)

IDEMPOTÊNCIA

Operação **idempotente**: executar várias vezes produz o **mesmo estado final no servidor** que executar uma vez.

Consultar filmes (GET) → mesma lista ✓ retry automático · Atualizar nome para "Ana" (PUT) → mesmo valor ✓ retry com cautela · Criar pedido (POST) → outro pedido criado ✗ sem retry · Confirmar pagamento → nova cobrança ✗ sem retry.

"O cliente viu uma falha. O servidor pode ter concluído a operação." → **Retry em mutation exige conhecer o contrato da API.**

Cache invalidation (quando dá certo)

PROBLEMA

A mutation funcionou, a API mudou, mas a tela continua velha. Porque **query e mutation são independentes**: a mutation não descobre sozinha quais queries foram afetadas.

Invalidar = declarar que o dado envelheceu. Ao invalidar: (1) a query é marcada como **stale**; (2) se estiver **ativa**, faz **refetch em segundo plano**. *cache antigo* → *stale* → *refetch* → *cache atualizado*. Invalidar **não apaga** o cache.

```
import { useQueryClient } from "@tanstack/react-query";

const queryClient = useQueryClient(); // acesso às queries do cache
const createTweetMutation = useMutation({
  mutationFn: createTweet,
  onSuccess: () => { // SÓ depois do sucesso (API já concluiu)
    queryClient.invalidateQueries({ queryKey: ["tweets"] }); // MESMA chave da query da lista
  },
});
```

Conceito	Por que acontece	O que faz
Retry	uma operação falhou	repete a operação
Refetch	dados precisam ser consultados de novo	executa outra busca
Invalidação	o cache deixou de ser confiável	marca stale e pode provocar refetch

PROBLEMAS COMUNS (LISTA DO PROFESSOR)

Ativar retry em qualquer mutation · esperar que retry corrija dados inválidos · esperar que a mutation atualize todas as queries · invalidar **antes** de a API concluir · usar **queryKey diferente** da query afetada · confundir invalidação com **exclusão** do cache.

B5. Aula 05 — Organizando o acesso a dados remotos

PROBLEMA

A tela conhece detalhes demais: queryKey, queryFn, mutationFn, retry, invalidação, eventos e UI — **responsabilidades diferentes no mesmo lugar**. Solução: separar em camadas.

Camada	Função principal	O que conhece
Serviço (services/)	Transportar os dados	endpoint, método HTTP, payload, formato da resposta (ou chamadas do SDK Firebase)
Custom Hook (hooks/)	Gerenciar o ciclo remoto	identidade do cache (queryKey), função de consulta, políticas (staleTime, retry), estado da operação, invalidação, integração com a biblioteca
Tela (app/)	Declarar intenções e apresentar	loading/erro na UI, alertas, navegação, eventos do usuário, componentes visuais

```
src/
├── app/tweets.tsx           ← tela
├── hooks/
│   ├── useTweetsQuery.ts   ← 1 hook por arquivo, nome = intenção
│   ├── useCreateTweetMutation.ts
│   ├── useUpdateTweetMutation.ts
│   └── useDeleteTweetMutation.ts
└── services/tweets.ts      ← getTweets, createTweet, updateTweet, deleteTweet
```

hooks/useTweetsQuery.ts

```
import { useQuery } from "@tanstack/react-query";
import { getTweets } from "@services/tweets";

export function useTweetsQuery() {
  return useQuery({ queryKey: ["tweets"], queryFn: getTweets });
}
```

hooks/useCreateTweetMutation.ts

```
export function useCreateTweetMutation() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: createTweet,
    retry: false,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["tweets"] }), // invalidação PERTO da alteração
  });
}
```

hooks/useDeleteTweetMutation.ts

```
export function useDeleteTweetMutation() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: number) => deleteTweet(id),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["tweets"] }),
  });
}
```

```
export default function TweetsScreen() {
  const tweetsQuery = useTweetsQuery();
  const createTweetMutation = useCreateTweetMutation();
  const deleteTweetMutation = useDeleteTweetMutation();

  function handlePublish(tweet: NewTweet) { createTweetMutation.mutate(tweet); }
  // ... if (tweetsQuery.isPending) ... FlatList ... onPress={() => deleteTweetMutation.mutate(item.id)}
}
```

- **Custom hook compõe hooks existentes** — não cria novo mecanismo de estado. **Abstrair é elevar a linguagem da aplicação:** `useTweetsQuery()` comunica intenção; o ganho não é nº de linhas.
- **Contrato:** a tela recebe `data`, `isPending`, `isError`, `isSuccess`, `refetch`; a configuração interna pode mudar sem mudar a tela.
- **Custom hooks reutilizam LÓGICA, não criam estado global.** Quem compartilha o DADO é o TanStack Query (`queryKey` + `cache`).
- Ponto único de configuração → menos divergência (`queryKey`/`staleTime`/`retry` iguais em todas as telas) → comportamento previsível. A abstração permanece **fin**a.

REGRAS DOS HOOKS

Nome começa com `use` · chamar **no corpo do componente** (topo) · **nunca** dentro de `if`, `loop` ou evento (`onPress`) · usar só em componente ou em outro hook · responsabilidade declarada pelo nome.

ERRADO: `onPress={() => useDeleteTweetMutation().mutate(id)}` · CERTO: criar no topo e chamar `.mutate(id)` no evento.

PARTE C — Identidade: autenticação, sessão, JWT, Firebase Auth

C1. Aula 06 — Autenticação, sessão e JWT

PROBLEMA

Sem identificação a API não distingue Ana de Bruno. **Um identificador não comprova identidade** (?userId=1 qualquer um envia). A API precisa **confirmar** quem está por trás → credenciais (usuário + senha) → POST /auth/login → aceita ou recusa. Depois, **não reenviar a senha** em cada requisição: guardar o resultado (token) e usá-lo nas próximas.

Conceito	Responsabilidade	Frase da síntese
Autenticação	Confirmar quem é o usuário (credenciais no login)	login é um evento
Sessão	Manter o estado autenticado entre as requisições	sessão é um estado
JWT	Formato compacto de token que carrega claims (informações)	JWT é uma credencial
Autorização	"O que você pode fazer?" (≠ autenticação "Quem é você?")	a API valida a identidade

autenticação ≠ sessão ≠ JWT · nem toda autenticação usa JWT · nem todo token é JWT.

JWT = JSON WEB TOKEN: HEADER.PAYLOAD.SIGNATURE

Header: algoritmo e tipo · **Payload:** claims (ex.: id, expiração) · **Signature:** proteção de **integridade**.
Header e payload são **Base64URL = CODIFICADOS, NÃO CRIPTOGRAFADOS** → qualquer um com o token lê. **Nunca** colocar senha/segedos no payload. Ler o token não prova que é verdadeiro — a **API verifica assinatura e validade**.
Bearer token = "credencial ao portador": quem possui, usa. Por isso: proteger o token, usar **HTTPS**, não logar o token.

usuário + senha → POST /auth/login → accessToken → sessão → header Authorization: Bearer <token> → API

Operação	Recurso TanStack	Motivo
Entrar (login)	useMutation	Operação imperativa, disparada após envio do formulário (mesmo sem alterar recurso de negócio)
Buscar o perfil	useQuery	Dado remoto consultável
Sair	Sessão (estado)	Remove a credencial mantida pelo app

services/auth.ts

```
export async function login(credentials: { username: string; password: string }) {
  const response = await fetch("https://dummyjson.com/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(credentials),
  });
  if (!response.ok) throw new Error("Usuário ou senha inválidos");
  return response.json(); // { id, firstName, email, accessToken, refreshToken }
}

export async function getUserProfile(accessToken: string) {
  const response = await fetch("https://dummyjson.com/auth/me", {
    headers: { Authorization: `Bearer ${accessToken}` }, // consulta protegida
  });
  if (!response.ok) throw new Error("Sessão inválida"); // sem token/token inválido → API recusa (401)
  return response.json();
}
```

providers/SessaoProvider.tsx (sessão compartilhada via Context)

```
type Sessao = { userId: number; accessToken: string }; // SEM senha!

const SessaoContext = createContext<{ sessao: Sessao | null; criarSessao: (s: Sessao) => void; sair: () => void } | null>(null);

export function SessaoProvider({ children }: { children: React.ReactNode }) {
  const [sessao, setSessao] = useState<Sessao | null>(null);
  const criarSessao = (nova: Sessao) => setSessao(nova);
  const sair = () => setSessao(null);
  return <SessaoContext.Provider value={{ sessao, criarSessao, sair }}>{children}</SessaoContext.Provider>;
}

export const useSessao = () => useContext(SessaoContext)!;
```

hooks/useLoginMutation.ts + hooks/usePerfilQuery.ts

```
export const useLoginMutation = () => useMutation({ mutationFn: login });

export function usePerfilQuery() {
  const { sessao } = useSessao();
  return useQuery({
    queryKey: ["perfil", sessao?.userId], // userId na chave; o TOKEN não precisa
    queryFn: () => getUserProfile(sessao?.accessToken ?? ""), // só executa se houver sessão
    enabled: sessao !== null,
  });
}
```

app/login.tsx

```
const loginMutation = useLoginMutation();
const { criarSessao } = useSessao();

function handleLogin() {
  loginMutation.mutate(
    { username, password },
    {
      onSuccess: ({ id, accessToken }) => {
        criarSessao({ userId: id, accessToken }); // inicia a sessão APÓS o sucesso
        router.redirect("/");
      },
    },
  );
}

// botão: disabled={loginMutation.isPending}; erro: loginMutation.error?.message
```

app/index.tsx (a sessão coordena a interface)

```
const { sessao } = useSessao();
if (!sessao) return <Redirect href="/login" />;
return <Redirect href="/perfil" />;
```

PEGADINHAS

- `enabled` evita chamar a API sem token.
- Senha nunca vai para a sessão.
- `useState` some ao fechar o app; para persistir o token use-se **Expo SecureStore** (armazenamento seguro – citado nas referências), nunca `AsyncStorage` para segredo.
- No slide há um typo (`setSession` vs `setSessao`; `sessao?.token` vs `accessToken`) — use nomes consistentes.
- Refresh token existe, mas a aula usa só o `accessToken`.

C2. Aula 07 — Protegendo o aplicativo (Firebase Auth + rotas protegidas)

IDEIA

Nem toda tela é pública. O **estado de autenticação decide quais rotas ficam disponíveis**. Sem usuário: rotas públicas. Com usuário: públicas + privadas. Princípio do **menor privilégio** (Saltzer & Schroeder).

App da aula: **PaceUp** (corridas de rua): eventos e detalhes são públicos; "Minhas inscrições" exige login.

Firebase Authentication = serviço de identidade: cria usuários, autentica credenciais, **mantém a sessão** e **informa mudanças** no estado de autenticação (delegar a parte "commodity": recuperação de acesso, verificação, persistência).

Setup no Console

Conta Google → Firebase Console → **criar projeto** → **registrar app Web** (gera o `firebaseConfig`, que *identifica* o projeto) → Authentication → Sign-in method → **habilitar Email/Password** (criar projeto não ativa login sozinho).

```
npx expo install firebase # Firebase JavaScript SDK
```

lib/firebase.ts – configura o Firebase

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore"; // aula 08+

const firebaseConfig = { apiKey: "...", authDomain: "...", projectId: "...", appId: "..." };

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app); // aula 08+
```

Função (firebase/auth)	O que faz
<code>createUserWithEmailAndPassword(auth, email, senha)</code>	cria conta e já autentica
<code>signInWithEmailAndPassword(auth, email, senha)</code>	entra com conta existente
<code>signOut(auth)</code>	sai → listener recebe null
<code>onAuthStateChanged(auth, (user) => ...)</code>	observa o estado: recebe <code>User</code> ou <code>null</code> ; retorna função de unsubscribe

providers/AuthProvider.tsx – mantém o estado da autenticação

```
type AuthContextValue = {
  user: User | null; isLoading: boolean;
  signUp: (e: string, p: string) => Promise<void>;
  signIn: (e: string, p: string) => Promise<void>;
  signOut: () => Promise<void>;
};
const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true); // ao abrir, a sessão AINDA está sendo verificada

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (u) => {
      setUser(u);
      setIsLoading(false);
    });
    return unsubscribe; // cleanup ao desmontar
  }, []);

  const value = {
    user, isLoading,
    signUp: async (e, p) => { await createUserWithEmailAndPassword(auth, e, p); },
    signIn: async (e, p) => { await signInWithEmailAndPassword(auth, e, p); },
    signOut: () => signOut(auth), // (importe signOut como fbSignOut p/ evitar conflito de nome)
  };
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
export const useAuth = () => useContext(AuthContext)!;
```

app/_layout.tsx – protege as rotas

```
function RootNavigator() {
  const { user, isLoading } = useAuth();
  if (isLoading) return <ActivityIndicator />; // não decidir rotas antes de saber se há sessão
  return (
    <Stack>
      <Stack.Screen name="index" /> // /* público */
      <Stack.Screen name="event/[id]" /> // /* público (rota dinâmica) */
      <Stack.Protected guard={!user}> // /* só SEM usuário */
        <Stack.Screen name="sign-up" />
        <Stack.Screen name="sign-in" />
      </Stack.Protected>
      <Stack.Protected guard{!!user}> // /* só COM usuário */
        <Stack.Screen name="registrations" />
      </Stack.Protected>
    </Stack>
  );
}
export default function Layout() {
  return (<AuthProvider><QueryClientProvider client={queryClient}><RootNavigator /></QueryClientProvider></AuthProvider>);
}
```

criar conta → Firebase Auth → onAuthStateChanged recebe User → AuthProvider atualiza user → Stack.Protected libera registrations
signOut() → onAuthStateChanged recebe null → registrations volta a ficar protegida

Arquivo	Responsabilidade
lib/firebase.ts	configura o Firebase (initializeApp, getAuth, getFirestore)
providers/AuthProvider.tsx	mantém o estado da autenticação (user, isLoading, signUp, signIn, signOut)
app/_layout.tsx	protege as rotas (Stack.Protected)
app/sign-up.tsx, app/sign-in.tsx	executam a autenticação (formulário chama signUp/signIn)

PEGADINHAS

- guard={!user} — !! converte para booleano.
- Esconder rota no app **não protege os dados** no servidor — isso é papel das **Security Rules** (aula 09).
- firebaseConfig (apiKey) **não é segredo** e **não autoriza** nada; só identifica o projeto.
- Você não navega manualmente após o login: o guard muda e o Expo Router redireciona.

PARTE D — Cloud Firestore como backend (BaaS)

D1. Aula 08 — Nem todo app precisa de uma API própria

CONCEITOS

Capacidades específicas (diferencial do produto): regras de domínio, fluxos próprios, cálculos exclusivos, decisões de negócio, integrações particulares.

Capacidades recorrentes (viraram **commodity** = comum e disponível, não "simples"): identificação de usuários, persistência, arquivos, controle de acesso, notificações.

Construir algo próprio custa além da 1ª versão: implantar, **operar/monitorar**, corrigir/atualizar dependências, escalar e controlar custos.

BaaS (Backend as a Service): plataforma que fornece capacidades de backend via **SDKs e APIs** (identidade, banco, arquivos, funções, eventos, notificações). O SDK roda no app e **conecta** ao serviço remoto que roda na infraestrutura do provedor. **O backend continua existindo** — muda *quem* constrói/opera.

Plataforma faz	Equipe da aplicação faz	Limites (compromissos) do BaaS
executar os serviços, manter infra disponível, atualizar plataforma, oferecer escalabilidade, persistir dados	definir modelo de dados, regras de domínio, configurar controle de acesso (Rules), acompanhar consumo/custos, validar dados	dependência da plataforma (lock-in) · consultas limitadas ao modelo e índices · cobrança por uso · migração trabalhosa · precisa configurar bem as regras de acesso

BaaS e API própria podem coexistir (escolha capacidade por capacidade): BaaS p/ autenticação, armazenamento, dados com acesso direto; API própria p/ regras exclusivas, integrações privadas e operações privilegiadas.

Elemento	Responsabilidade no acesso direto
Configuração do Firebase	identificar projeto e aplicação
Firebase Authentication	identificar o usuário atual
Security Rules	autorizar ou rejeitar a operação
Cloud Firestore	executar a operação permitida

Modelo orientado a documentos

posts ← COLEÇÃO (agrupa documentos)
└─ 7qA2fLm9 ← DOCUMENTO (identificador) caminho completo: posts/7qA2fLm9
 ├─ content: "Estudando bancos NoSQL" ← CAMPOS
 ├─ authorId: "user-42"
 └─ createdAt: Timestamp

Relacional	Documentos (Firestore)
Tabelas organizam linhas	Coleções organizam documentos
Colunas definem estrutura	Campos representam valores
Chave primária identifica a linha	Caminho identifica o documento
Relacionamentos/JOINS	Documentos podem conter mapas e listas
O banco aplica o esquema	A aplicação controla a consistência (sem esquema rígido → mais validação)

Dois documentos da mesma coleção não podem ter o mesmo ID. **O modelo considera as consultas**: "recentes" exige `createdAt`; "por autor" exige `authorId`; ordenar exige campo consultável e **indexado**.

Lendo: referência → consulta → execução → resultado

```
services/posts.ts

import { collection, query, orderBy, limit, getDocs, Timestamp } from "firebase/firestore";
import { z } from "zod";
import { db } from "@lib/firebase";

const postsCollection = collection(db, "posts"); // ONDE? (só referência, nada foi lido)
const recentPostsQuery = query( // QUAIS? (descrever ≠ executar)
  postsCollection,
  orderBy("createdAt", "desc"), // EM QUE ORDEM?
  limit(10), // QUANTOS?
);

const postSchema = z.object({ // contrato esperado (validação em execução)
  id: z.string(),
  content: z.string(),
  authorId: z.string(),
  createdAt: z.instanceof(Timestamp),
});
export type Post = z.infer<typeof postSchema>;

export const mapPost = (document) =>
  postSchema.parse({ ...document.data(), id: document.id }); // id vem do snapshot, campos de data()

export async function getRecentPosts(): Promise<Post[]> {
  const snapshot = await getDocs(recentPostsQuery); // QUANDO? executa → QuerySnapshot
  return snapshot.docs.map(mapPost);
}
```

```
hooks/useRecentPostsQuery.ts + tela

export function useRecentPostsQuery() {
  return useQuery({ queryKey: ["posts", "recent"], queryFn: getRecentPosts });
}
// tela:
const { data: posts, isFetching, refetch } = useRecentPostsQuery();
<FlatList data={posts} refreshing={isFetching} onRefresh={() => refetch()} ... /> // "puxar p/ atualizar"
```

Elemento	Pergunta respondida
collection()	Onde estão os documentos?
query()	Quais documentos queremos?
orderBy() / limit()	Em qual ordem? / Quantos?
getDocs()	Quando executar a leitura?
QuerySnapshot	Qual foi o resultado? → .empty (nenhum doc?), .size (quantidade), .docs (lista de QueryDocumentSnapshot)

SNAPSHOT = FOTOGRAFIA

getDocs termina depois de produzir o resultado. Se outro cliente criar um doc depois, o snapshot anterior **não muda**; só uma nova execução (refetch, remontagem stale, foco/reconexão) gera outro snapshot. Assinatura contínua = aula 10.

Tecnologia	Responsabilidade (as duas usam a palavra "query"!)
Firestore (query())	descrever e executar a seleção remota
TanStack Query (useQuery)	controlar execução, cache e estados
Zod	validar os documentos recebidos
React Native	apresentar o resultado

PEGADINHA TYPESCRIPT

document.data() as Post só "informa o compilador" — **não verifica nada** em execução. Documentos podem ter campos ausentes, tipos diferentes, data como texto. Por isso **Zod**: schema.parse() transforma documento inválido em **falha tratável** (vira error no useQuery).

D2. Aula 09 — Salvando dados em um BaaS (addDoc)

Com API própria	Com Cloud Firestore
A equipe implementa o endpoint	O SDK acessa um serviço existente
A API recebe a solicitação	O Firestore recebe
O servidor valida a autorização	Security Rules validam
O servidor persiste	O Firestore persiste o documento

"O SDK descreve e transporta a operação. O serviço remoto decide e persiste." Destino, contrato, identidade e falhas continuam existindo.

CONTRATO DO ADDDOC

`addDoc(CollectionReference, dados): Promise<DocumentReference>`

CollectionReference = em qual coleção? · data = quais campos? · Promise = quando termina? · DocumentReference = onde ficou o doc criado?

- **ID automático**: cada `addDoc` gera um novo ID (o app não escolhe). O ID pertence ao **caminho**, **não** aos campos (`data()` não traz o id).
- **ID automático não representa ordem** → precisa de campo `createdAt` para `orderBy`.
- **`new Date()` é ruim**: relógio do dispositivo pode estar errado/alterado/discordar. Use `serverTimestamp()` : devolve um **FieldValue especial** (instrução); o **servidor** resolve para `Timestamp`.
- **DocumentReference ≠ snapshot**: tem `.id` e `.path` ("posts/7qA2fLm9"), mas **NÃO** tem os campos, nem o `createdAt` final, nem a coleção atualizada, nem o cache do TanStack atualizado.

services/posts.ts (criação)

```
import { addDoc, collection, serverTimestamp } from "firebase/firestore";

type NewPost = { content: string; authorId: string };

export async function createPost(input: NewPost) {
  return addDoc(postsCollection, {
    ...input,
    createdAt: serverTimestamp(), // horário decidido pelo servidor
  }); // resolve com DocumentReference
}
```

hooks/useCreatePostMutation.ts

```
export function useCreatePostMutation() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: createPost, // mutation NÃO depende de HTTP: só precisa retornar Promise
    retry: false, // addDoc repetido = documento DUPLICADO (novo ID)
    onSuccess: async (documentReference) => {
      console.log(documentReference.id);
      await queryClient.invalidateQueries({ queryKey: ["posts"] }); // ["posts"] invalida também ["posts","recent"] (prefixo)
    },
  });
}
// tela: createPostMutation.mutate({ content, authorId: user.uid })
```

PROMISE DO ADDDOC E "DOIS CACHES"

Resolve quando o **backend confirma**; rejeita quando o backend informa falha; pode ficar **pendente** sem conexão (isPending acompanha isso).

Compensação de latência: o SDK aplica a escrita no **estado local** antes da confirmação remota (aplicação local ≠ confirmação).

Existem dois caches: cache do cliente Firestore (estado local do SDK) ≠ cache do TanStack Query (valor da `queryFn`). Depois do `addDoc`: Firestore 11 docs | TanStack 10 → por isso **invalidateQueries**. Invalidação não persiste nada; para query ativa, gera outro `getDocs`.

Security Rules — quem autoriza?

O cliente não é uma fronteira confiável: um cliente modificado pode ignorar validação do formulário, alterar campos, trocar o `authorId`, chamar o SDK fora do fluxo. O `firebaseConfig` identifica, **não autoriza**. A autorização acontece no serviço, pelas **Security Rules**, avaliadas **antes da persistência**.

Item das Rules	Significado
<code>get, list, create, update, delete</code>	operações distintas (<code>read</code> = <code>get+list</code> ; <code>write</code> = <code>create+update+delete</code>)
<code>request.auth / request.auth.uid</code>	identidade do Firebase Auth (null se não logado)
<code>request.resource.data</code>	estado proposto : campos que o cliente tenta gravar
<code>resource.data</code>	estado atual do documento já salvo (útil em <code>update/delete</code>)
<code>keys().hasAll([...])</code>	exige os campos obrigatórios
<code>keys().hasOnly([...])</code>	rejeita campos não previstos
<code>is string, .size()</code>	tipo e tamanho
<code>request.time</code>	horário do servidor da requisição (compara com <code>serverTimestamp</code>)

firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function hasExpectedFields() {
      return request.resource.data.keys().hasAll(["content", "authorId", "createdAt"])
        && request.resource.data.keys().hasOnly(["content", "authorId", "createdAt"]);
    }
    function isValidContent() {
      return request.resource.data.content is string
        && request.resource.data.content.size() > 0
        && request.resource.data.content.size() <= 280;
    }

    match /posts/{postId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && hasExpectedFields()
        && isValidContent()
        && request.resource.data.authorId == request.auth.uid // autor = quem está logado
        && request.resource.data.createdAt == request.time; // horário veio do servidor
      // (extra, p/ aula 11) só o autor altera/exclui:
      allow update, delete: if request.auth != null && resource.data.authorId == request.auth.uid;
    }
  }
}
```

Mecanismo	Responsabilidade
Formulário	feedback antes do envio (UX)
TypeScript	coerência de tipos durante o desenvolvimento
Security Rules	autorização e proteção do dado no serviço remoto

"Validação no cliente melhora o uso. Regra no backend protege o dado." Falhas possíveis do addDoc: não autenticado, regras recusaram, serviço inalcançável, limite do projeto.

```
mutate(input) → createPost(input) → addDoc(postsCollection, data) → Security Rules
├─ negam → Promise rejeita → onError / isError
└─ permitem → documento criado → DocumentReference → onSuccess → invalidateQueries(["posts"]) → novo getDocs
```

D3. Aula 10 — Dados em tempo real: `onSnapshot`

PROBLEMA

Celular A publica; celular B continua desatualizado. Leitura pontual (`getDocs`) = fotografia, não recebe mudanças futuras. Para acompanhar, o cliente precisa manter uma **assinatura** (listener orientado a eventos). **A consulta é a mesma; muda o modo de consumo.**

	<code>getDocs(q)</code>	<code>onSnapshot(q, onNext, onError)</code>
Necessidade	carregar uma vez	acompanhar mudanças enquanto a tela estiver ativa
Contrato	pedir → aguardar → receber → terminar	assinar → receber → receber → ... → cancelar
Retorno	<code>Promise<QuerySnapshot></code>	Unsubscribe (função) — NÃO devolve docs nem Promise
Dados chegam	no <code>await</code>	no callback <code>onNext</code> (1ª emissão = estado atual; depois a cada add/modify/remove)

- Primeira emissão:** ao registrar, o callback já recebe o estado atual → **não precisa de `getDocs` antes.**
- Erro** (permissão insuficiente, consulta inválida) chega no 3º argumento e **encerra o listener** (não emite mais).
- Cleanup:** setup (assinar) e cleanup (`unsubscribe()`) formam uma única responsabilidade. Listener sem cleanup = contrato incompleto (vazamento de recursos/cobrança de leituras).
- Montagem x foco:** numa pilha de navegação a tela anterior continua **montada** quando outra fica visível. Cleanup no unmount (`useEffect`) manteria o listener ativo. Por isso: **`useFocusEffect`** — assina ao **ganhar foco**, cancela ao **perder foco**.

hooks/useRecentPostsSubscription.ts

```
import { useFocusEffect } from "expo-router";
import { useCallback, useState } from "react";
import { onSnapshot } from "firebase/firestore";

export function useRecentPostsSubscription() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [error, setError] = useState<unknown>(null);
  const [isLoading, setIsLoading] = useState(true);

  useFocusEffect(
    useCallback(() => {
      setError(null);
      setIsLoading(true);
      return onSnapshot(
        recentPostsQuery,
        (snapshot) => { setPosts(snapshot.docs.map(mapPost)); setIsLoading(false); },
        (cause) => { setError(cause); setIsLoading(false); },
      );
    }, []),
  );
  return { posts, error, isLoading };
}
```

app/community.tsx (tela não conhece onSnapshot)

```
const { posts, error, isLoading } = useRecentPostsSubscription();
if (isLoading) return <LoadingState />;
if (error) return <ErrorState />;
return <PostList posts={posts} />;
```

Síntese: uma assinatura tem **começo, emissões, falha e encerramento** — ignorar qualquer etapa gera interface inconsistente.

D4. Aula 11 — Atualizando e excluindo documentos

Necessidade	Operação (CRUD)	Alvo	API
registrar informação nova (passa a existir)	CREATE	coleção	addDoc
conhecer informações (chega à interface)	READ	coleção / consulta	getDocs / onSnapshot
mudar parte (identidade permanece, estado muda)	UPDATE	documento	updateDoc
retirar (identidade deixa de existir)	DELETE	documento	deleteDoc

A OPERAÇÃO PRECISA DE UM ALVO ESPECÍFICO

Título e posição na lista NÃO são identidade (título repete/muda; posição muda com ordenação/inserção). Identidade = **doc.id**, que forma o caminho com o nome da coleção.

Fluxo do id: taskDocument (snapshot) → doc.id preservado no item ({...data(), id}) → task.id acompanha o clique → doc(db, "tasks", taskId) reconstrói o caminho → updateDoc / deleteDoc.

collection(db, "tasks") = **CollectionReference** /tasks (conjunto) · doc(db, "tasks", id) = **DocumentReference** /tasks/task-42 (um específico). Construir referência não lê nem escreve.

services/tasks.ts

```
import { doc, updateDoc, deleteDoc } from "firebase/firestore";

export async function completeTask(taskId: string): Promise<void> {
  const taskReference = doc(db, "tasks", taskId);
  await updateDoc(taskReference, { completed: true }); // PARCIAL: só "completed" muda; title/subject preservados
  // updateDoc FALHA se o documento não existir
}

export async function removeTask(taskId: string): Promise<void> {
  const taskReference = doc(db, "tasks", taskId);
  await deleteDoc(taskReference); // remove doc inteiro (não sobra doc "vazio")
  // subcoleções NÃO são removidas junto
}
```

hooks/useCompleteTaskMutation.ts / useRemoveTaskMutation.ts

```
export const useCompleteTaskMutation = () => useMutation({ mutationFn: completeTask });
export const useRemoveTaskMutation = () => useMutation({ mutationFn: removeTask });
// Com LISTENER (onSnapshot) a lista se atualiza sozinha → não precisa invalidar.
// Com LEITURA PONTUAL (useQuery+getDocs) → onSuccess: invalidateQueries({ queryKey: ["tasks"] }).
```

```

const complete = useCompleteTaskMutation();
const remove = useRemoveTaskMutation();

function handleRemove() {
  Alert.alert("Excluir atividade?", "Essa ação não pode ser desfeita.", [ // confirmação: ação destrutiva sem lixeira
    { text: "Cancelar", style: "cancel" },
    { text: "Excluir", style: "destructive", onPress: () => remove.mutate(task.id) },
  ]);
}
<Pressable disabled={complete.isPending} onPress={() => complete.mutate(task.id)}>
  <Text>{complete.isPending ? "Salvando..." : "Concluir"}</Text>
</Pressable>
<Pressable disabled={remove.isPending} onPress={handleRemove}><Text>Excluir</Text></Pressable>
{(complete.isError || remove.isError) && <Text>Falha ao salvar. Tente novamente.</Text>}

```

PEGADINHAS DA AULA 11

- **Esconder o card não exclui o documento.** Evidências: filtrar estado local → só some da tela; `await deleteDoc` sem falha → servidor confirmou; conferir no Console → não existe mais. **A confirmação vem da Promise da operação.**
- Com `onSnapshot` a lista pode mudar **antes** da Promise resolver (compensação de latência / escrita local pendente) — a mutation continua `isPending` até o servidor confirmar. Sem conexão a Promise pode ficar pendente.
- Rejeitada → informar a falha; **não fingir sucesso remoto.**
- **Identificar o caminho não concede permissão:** 1. identificar (caminho) 2. solicitar (cliente expressa intenção) 3. autorizar (Security Rules). **Esconder um botão não protege o dado.**

PARTE E — Tabelas "NÃO CONFUNDA"

Par	Diferença em uma linha
useQuery × useMutation	Query lê e executa sozinha (observa dados, tem cache/queryKey). Mutation muda e só executa quando você chama <code>mutate()</code> .
queryKey × queryFn	queryKey identifica (cache); queryFn busca .
mutationFn × mutate	mutationFn = o que executar; mutate = quando executar.
mutationFn × onSuccess	mutationFn executa a operação; callbacks reagem ao resultado (limpar form, alertar, navegar, invalidar).
onSuccess × onError × onSettled	só sucesso · só erro · sempre no fim (sucesso ou erro).
isPending × isFetching	isPending (query) = ainda não tem dado . isFetching = há uma busca rodando (inclui refetch em 2º plano, quando já tem dado).
isPending (query) × isPending (mutation)	Query: aguardando 1º dado. Mutation: " esta operação ainda está sendo executada".
fresh × stale	Fresh = confiável, reutiliza sem buscar. Stale = continua no cache mas pode ser refeito.
staleTime × gcTime	staleTime: quanto tempo fica fresh (padrão 0). gcTime: quanto tempo uma query inativa fica no cache (padrão 5 min).
active × inactive	Há ≥ 1 componente usando × nenhum usando (dados continuam no cache até o gcTime).
retry × refetch × invalidação	Falhou → repete · precisa de dado novo → busca outra vez · cache não confiável → marca stale (+ refetch se ativa).
invalidar × remover cache	Invalidar só marca stale; não apaga os dados.
retry query × retry mutation	Padrão: query 3 tentativas extras; mutation 0. Mutation só com retry se a operação for idempotente .
Serviço × custom hook × tela	Transporta dado · gerencia ciclo remoto · declara intenção e apresenta.
Custom hook × estado global	Hook compartilha a forma de acessar; o dado é compartilhado pelo cache via queryKey.
Autenticação × autorização	"Quem é você?" × "O que você pode fazer?".
Autenticação × sessão × JWT	Confirmar identidade (evento) · manter estado autenticado (estado) · formato de credencial.
Codificado × criptografado	JWT (header/payload) é Base64URL: qualquer um decodifica. Não é segredo.
Identificador × identidade	?userId=1 qualquer um envia; identidade exige verificação (credenciais/token).
signUp × signIn	createUserWithEmailAndPassword cria e autentica; signInWithEmailAndPassword usa conta existente.
firebaseConfig × Security Rules	Config identifica o projeto; Rules autorizam .
Rota protegida × Security Rules	Stack.Protected controla navegação (UX); Rules protegem o dado no servidor.
request.resource.data × resource.data	Dados propostos pela escrita × dados atuais já salvos.
hasAll × hasOnly	Exige todos os obrigatórios × proíbe campos extras.
allow create × allow write	create só criação; write = create + update + delete.
Firestore query() × useQuery	Descreve/executa seleção remota × controla execução, cache e estados.
collection() × doc()	CollectionReference (conjunto, p/ addDoc/consultas) × DocumentReference (um doc, p/ updateDoc/deleteDoc).
DocumentReference × DocumentSnapshot	Referência aponta (id, path); snapshot contém o estado lido (data()).
document.id × document.data()	Identificador (vem do caminho) × campos armazenados (sem o id).
getDocs × onSnapshot	Fotografia pontual (Promise) × assinatura contínua (callbacks + Unsubscribe).
addDoc × updateDoc × deleteDoc	Cria com ID automático na coleção × altera campos de um doc existente (parcial, falha se não existe) × remove doc inteiro.
new Date() × serverTimestamp()	Relógio do dispositivo (não confiável) × instrução p/ o servidor definir o Timestamp.
as Post × schema.parse	Só engana o compilador × valida em tempo de execução (Zod).
Cache Firestore × cache TanStack	Estado local do SDK × valor retornado pela queryFn — não se sincronizam sozinhos.
Aplicação local × confirmação	SDK mostra a escrita antes (compensação de latência); confirmação = Promise resolvida.
useEffect × useFocusEffect	Setup/cleanup na montagem/desmontagem × no ganho/perda de foco da rota.
Nativo × cross × híbrido	1 código por SO · 1 código p/ ambos com componentes nativos (RN) · web em WebView.
justifyContent × alignItems	Eixo principal × eixo cruzado (no RN o principal é vertical: column).
44pt × 48dp	Área de toque mínima iOS (HIG) × Android (Material).

Códigos HTTP que aparecem na prova

Método	Uso	Idempotente?
GET	ler (query)	Sim
POST	criar (mutation) · login	Não (cria de novo)
PUT	substituir recurso	Sim
PATCH	alterar parte	Depende (ex.: "nome = Ana" sim; "saldo + 10" não)
DELETE	excluir	Sim (estado final: não existe)

Status: 200 OK · 201 Created · 400 dados inválidos · 401 não autenticado (sem/ token inválido) · 403 sem permissão · 404 não encontrado · 500 erro do servidor. `fetch` só rejeita em falha de rede — checar `response.ok`.

PARTE F — Projeto-modelo (onde cada coisa vai)

Se a prova pedir "implemente X", use este mapa: **primeiro decida a camada**, depois o código (exemplos nas partes B–D).

meu-app/ ├─ app/ │ ├── _layout.tsx │ ├── index.tsx │ ├── event/[id].tsx │ ├── sign-in.tsx / sign-up.tsx │ └── registrations.tsx ├─ components/ ├─ hooks/ │ ├── usePostsQuery.ts │ ├── useCreatePostMutation.ts │ ├── useUpdatePostMutation.ts / useDeletePostMutation.ts │ └── usePostsSubscription.ts ├─ services/ │ └── posts.ts ├─ providers/AuthProvider.tsx ├─ lib/firebase.ts └─ firestore.rules	 ← ROTAS (Expo Router: cada arquivo = uma tela) ← QueryClientProvider + AuthProvider + <Stack> + Stack.Protected ← lista (useXxxQuery ou useXxxSubscription), loading/erro/vazio ← rota dinâmica: const { id } = useLocalSearchParams() ← formulário → useAuth().signIn / signUp ← rota PRIVADA ← peças visuais (TaskItem, PostList, LoadingState, ErrorState) ← 1 hook por arquivo, nome = intenção ← useQuery({ queryKey, queryFn, staleTime, retry }) ← useMutation + retry:false + onSuccess invalidateQueries ← useFocusEffect + onSnapshot ← TRANSPORTE: fetch (API) ou SDK (getDocs, addDoc, updateDoc, deleteDoc) ← + schema Zod + mapPost ← user, isLoading, signUp, signIn, signOut (onAuthStateChanged) ← initializeApp, getAuth, getFirestore (export auth, db) ← autorização no servidor
---	---

Se a questão pede...	Arquivo	O que escrever
Configurar TanStack Query	app/_layout.tsx	new QueryClient() fora do componente + <QueryClientProvider client>
Listar dados de uma API	services + hooks + tela	função fetch com response.ok → useQuery → ifs isPending/error → FlatList
Evitar requisição ao voltar à tela	hook da query	staleTime: 30_000
Botão criar/excluir	hook da mutation + tela	useMutation, mutate(x), disabled={isPending}, feedback de erro
Lista não atualiza após criar	hook da mutation	onSuccess: () => queryClient.invalidateQueries({ queryKey: [mesma chave] })
Falha de rede na leitura	hook da query	retry: 2
Login com API (JWT)	services/auth + useLoginMutation + login.tsx	POST → onSuccess cria sessão {userId, accessToken} → redirect
Rota que exige token	service + hook	headers: { Authorization: `Bearer \${token}` } + enabled: sessao !== null
Login com Firebase	lib/firebase + AuthProvider	getAuth, signIn/createUser..., onAuthStateChanged, isLoading inicial true
Tela só para logado	app/_layout.tsx	<Stack.Protected guard={!user}>
Ler do Firestore	services	collection → query(orderBy, limit) → getDocs → docs.map({...data(), id}) (+ Zod)
Criar no Firestore	services + mutation	addDoc(col, {...input, createdAt: serverTimestamp()}) + invalidar
Proteger criação	firestore.rules	allow create: if request.auth !== null && ... authorId == request.auth.uid && createdAt == request.time
Tempo real	hook de assinatura	useFocusEffect(useCallback(() => onSnapshot(q, onNext, onError), []))
Editar/excluir doc	services + mutation + item	doc(db,"col",id) → updateDoc(ref,{campo}) / deleteDoc(ref) + Alert de confirmação

Commits sugeridos (Git Flow) se a prova for um projeto no GitHub Classroom

```
git checkout -b feature/tanstack-query-setup
feat: configura QueryClientProvider no layout raiz
feat(tweets): adiciona listagem com useTweetsQuery
feat(tweets): adiciona criação com useCreateTweetMutation e invalidação
refactor(tweets): extrai acesso remoto para services e hooks
feat(auth): adiciona AuthProvider com Firebase Auth e rotas protegidas
feat(posts): sincroniza mural com onSnapshot via useFocusEffect
fix(posts): valida documentos com zod
```

PARTE G — Banco de perguntas e respostas

Mistura de perguntas conceituais, de "complete o código", de "o que está errado" e dissertativas — no estilo dos "Pare e decida" e desafios das aulas.

Fundamentos (1º semestre)

1. Quais as diferenças entre desenvolvimento nativo, cross-platform e híbrido?

Nativo: um app por plataforma (Swift/Kotlin), melhor performance e acesso total ao hardware, mas 2× custo. Cross-platform: um código para iOS e Android (RN, Flutter), componentes nativos, reduz 40–60% do tempo, performance próxima. Híbrido: app web em WebView (Ionic/Cordova), rápido, mas performance e UX inferiores.

2. Por que React Native faz sentido para o curso/startups?

Um time, uma linguagem (TS), 70–90% de código compartilhado, componentes nativos de verdade, ecossistema Expo que dispensa configurar ambiente nativo.

3. Como o React Native funciona por baixo?

TS é transpilado para JS e empacotado (bundle) pelo Metro; no aparelho, a JSI comunica o JS com o código nativo; os componentes viram componentes nativos e a lógica fica em JS.

4. O que é Fast Refresh? E Metro?

Fast Refresh atualiza o app ao salvar sem rebuild, mantendo o estado quando possível. Metro é o bundler do React Native.

5. Qual versão do Node usar e por quê?

LTS — estável e com atualizações de segurança. Current tem novidades mas pode ser instável.

6. Equivalentes RN de div, p, button, img?

View, Text, Pressable/TouchableOpacity/Button, Image. Texto fora de <Text> quebra o app.

7. Qual o flexDirection padrão no RN e qual a consequência?

column (na web é row): os elementos empilham de cima para baixo. justifyContent age no eixo vertical e alignItems no horizontal.

8. Três Views com flex 1, 2 e 3. Quanto ocupa cada uma?

Espaço dividido em 6: 1/6, 2/6 (1/3) e 3/6 (1/2).

9. Como fazer sombra que funcione em iOS e Android?

iOS: shadowColor/shadowOffset/shadowOpacity/shadowRadius; Android: elevation. Unificar com Platform.select({ ios: {...}, android: {...} }).

10. Como fazer layout responsivo sem media queries?

useWindowDimensions() e renderização condicional (width > 600 ? <Tablet/> : <Celular/>).

11. Para que serve SafeAreaView?

Adicionar padding automático para o conteúdo não ficar atrás do notch, câmera, relógio ou barra inferior.

12. O que há de errado em {count && <Badge />} ?

Se count for 0, o React renderiza "0" (no RN, texto fora de Text pode quebrar). Correto: {count > 0 && <Badge />}.

13. O que significa "UI = f(props)" e por que props são imutáveis?

Componente deve ser função pura: mesmas props → mesmo JSX. Alterar props/variáveis externas torna o resultado imprevisível. Mutação local (de algo criado durante a renderização) é permitida.

14. O que é a prop children e como ajuda na performance?

Conteúdo passado entre as tags do componente. Passar um componente pesado como children de um wrapper que muda muito evita sua re-renderização, pois a prop não mudou.

15. Área mínima de toque em iOS e Android? Contraste mínimo?

44pt (HIG) e 48dp (Material); contraste 4.5:1 para texto pequeno.

16. Por que não usar cores hexadecimais fixas no Material 3?

Impedem o Dynamic Color e o dark mode; use tokens (colorPrimaryContainer, onSurface) / cores semânticas no iOS (systemRed, labelColor).

TanStack Query — leitura e cache (aulas 01–02)

17. O TanStack Query substitui o fetch? O que ele resolve?

Não. Ele organiza o estado produzido pela consulta (pending, data, error), além de cache, refetch e retry, tirando do componente o controle manual com useState/useEffect.

18. Quais as duas informações mínimas de uma query?

queryKey (qual consulta / identidade no cache) e queryFn (como buscar).

19. Onde e como configurar o TanStack Query num projeto Expo Router?

Em app/_layout.tsx: criar const queryClient = new QueryClient() fora do componente e envolver o <Stack /> com <QueryClientProvider client={queryClient}>.

20. Complete: const { data, ___, error } = useQuery({ queryKey: ___, queryFn: ___ })

isPending, ["movies"], fetchMovies (sem parênteses).

21. Qual o erro em queryFn: fetchMovies() ?

Chama a função imediatamente e passa uma Promise em vez da função; o TanStack precisa da referência para decidir quando executar.

22. Desafio: mostrar quantidade de tweets ou "Nenhum tweet encontrado."

Após tratar `isPending` e `error`: `if (data.length === 0) return <Text>Nenhum tweet encontrado.</Text>; return <Text>{data.length} tweets</Text>.`

23. Por que ao sair e voltar para a tela acontece nova requisição?

Dados estavam no cache, mas `stale` (`staleTime` padrão 0); quando o componente volta a usar a `query` surge uma oportunidade de `refetch`. Os dados antigos aparecem enquanto o novo chega.

24. Diferença entre `fresh` e `stale`?

`Fresh`: dado considerado atual, reutilizado sem buscar. `Stale`: continua no cache, mas pode ser buscado de novo na próxima oportunidade.

25. O que fazem `staleTime` e `gcTime`?

`staleTime`: por quanto tempo o dado permanece `fresh`. `gcTime`: por quanto tempo uma `query` inativa fica no cache antes da `garbage collection`.

26. Quando uma `query` `stale` faz `refetch`?

Quando um componente passa a usá-la (montagem), quando o app recupera o foco, quando a rede reconecta — ou chamando `refetch()` / invalidando.

27. `Active`, `inactive` e `garbage collected`?

`Active`: há componente usando. `Inactive`: nenhum usa, dados ficam no cache. `Garbage collected`: inativa além do `gcTime`, removida; nova consulta será necessária.

28. Dois componentes usam `queryKey: ["products"]`. Quantas requisições/dados?

Compartilham a mesma entrada de cache (mesmos dados); a `query` fica ativa enquanto algum deles a usar.

29. Configure: lista de produtos `fresh` por 1 minuto.

`useQuery({ queryKey: ["products"], queryFn: getProducts, staleTime: 60_000 })`.

30. Como inspecionar o cache no Expo?

`npx expo install @dev-plugins/react-query`, chamar `useReactQueryDevTools(queryClient)` e pressionar `shift+m` no terminal.

Mutations, retry e invalidação (aulas 03–04)

31. Qual a diferença entre `query` e `mutation`?

`Query` obtém dados (`GET`), executa ao ser usada. `Mutation` causa mudança (`POST/PUT/PATCH/DELETE`) e só executa quando `mutate()` é chamado.

32. Papel de `mutationFn` e de `mutate`?

`mutationFn`: o que executar (função assíncrona que fala com a API). `mutate`: quando executar, normalmente após ação do usuário, passando as variáveis.

33. Estados de uma `mutation`?

`isIdle` (não começou), `isPending` (executando), `isSuccess` (sucesso), `isError` (erro). Um por vez.

34. Como impedir que o usuário crie dois tweets clicando rápido?

`<Pressable disabled={isPending}>` e trocar o texto para "Salvando...".

35. Onde limpar o formulário após salvar? E mostrar alerta de erro?

`onSuccess: () => setText("");` `onError: () => Alert.alert(...)`. `onSettled` roda em ambos os casos.

36. Desafio: excluir um tweet. Requisitos?

Função `deleteTweet(id)` com `DELETE`; `useMutation({ mutationFn: deleteTweet })`; `mutate(item.id)`; `disabled={isPending}`; feedback em `isError`. Não atualizar a lista manualmente.

37. Comportamento padrão de `retry` de `query` e de `mutation`? Por quê?

`Query`: até 3 novas tentativas. `Mutation`: nenhuma — repetir uma mudança pode duplicar efeitos (o servidor pode ter concluído mesmo que o cliente veja falha).

38. `retry: 2` significa quantas execuções no total?

3 (1 inicial + 2 novas tentativas).

39. `Retry` resolve erro 404, 400 ou 401?

Não. `Retry` ajuda em falhas transitórias (conexão instável, `timeout`, servidor temporariamente fora), não em endereço errado, dados inválidos ou falta de autorização.

40. O que é operação `idempotente`? Dê exemplos.

Executar várias vezes produz o mesmo estado final que uma vez. `Idempotente`: consultar filmes, atualizar nome para "Ana", excluir. Não `idempotente`: criar pedido, confirmar pagamento, `addDoc`.

41. "Pare e decida": carregar lista de filmes, atualizar nome de um filme, criar pedido.

Carregar lista → `retry` automático. Atualizar nome → `retry` com cautela (`idempotente`, mas conferir contrato). Criar pedido → sem `retry`.

42. A `mutation` funcionou mas a lista não mudou. Por quê e como resolver?

`Query` e `mutation` são independentes; a `mutation` não sabe quais `queries` afetou. Solução: no `onSuccess`, `queryClient.invalidateQueries({ queryKey: ["tweets"] })` com a mesma chave da lista.

43. O que acontece ao invalidar uma `query`?

É marcada como `stale` e, se ativa, faz `refetch` em segundo plano: `cache antigo` → `stale` → `refetch` → `cache atualizado`. Os dados não são apagados.

44. Por que invalidar no `onSuccess` e não antes do `mutate`?

Invalidar antes de a API concluir faria o `refetch` buscar dados ainda sem a mudança.

45. Como obter o `queryClient` dentro de um componente/hook?

`const queryClient = useQueryClient();` (import de `@tanstack/react-query`).

46. Liste os "problemas comuns" da aula 04.

Retry em qualquer mutation; esperar que retry corrija dados inválidos; esperar que a mutation atualize todas as queries; invalidar antes da API concluir; queryKey diferente; confundir invalidação com exclusão do cache.

47. Diferencie `retry`, `refetch` e `invalidação`.

Retry: operação falhou → repete. Refetch: dados precisam ser consultados de novo → nova busca. Invalidação: cache deixou de ser confiável → marca stale e pode provocar refetch.

Arquitetura (aula 05)

48. Qual o problema de deixar `useQuery/useMutation` direto na tela?

A tela conhece detalhes demais (queryKey, queryFn, retry, invalidação, eventos e UI): responsabilidades diferentes no mesmo lugar, duplicação e divergência entre telas.

49. Responsabilidade do serviço × custom hook?

Serviço transporta os dados (endpoint, método HTTP, payload, formato da resposta). Custom hook gerencia o ciclo remoto (queryKey, queryFn, staleTime/retry, estado, invalidação, integração com a biblioteca).

50. Um custom hook cria estado global?

Não. Reutiliza lógica (a forma de acessar). O dado é compartilhado pelo TanStack Query via queryKey e cache.

51. Qual o ganho de `useTweetsQuery()` se a quantidade de linhas é parecida?

Eleva a linguagem da aplicação (intenção em vez de detalhe técnico), estabelece um contrato estável para a tela e centraliza a configuração → comportamento previsível.

52. Onde fica a invalidação do cache após criar um tweet?

Dentro do `useCreateTweetMutation`, no `onSuccess` — "a mutation mantém a invalidação próxima da alteração".

53. Por que criar uma mutation por ação (`create/update/delete`)?

Cada ação tem função, estado (isPending etc.) e efeitos posteriores independentes.

54. O que está errado: `onPress={() => useDeleteTweetMutation().mutate(id)}`?

Viola as regras dos hooks (chamado dentro de evento). Chamar o hook no corpo do componente e usar `.mutate(id)` no evento.

55. Cite as regras dos hooks.

Nome começa com use; chamar no corpo do componente; nunca em condição, loop ou evento; usar só em componente ou outro hook; responsabilidade declarada pelo nome.

Autenticação e JWT (aula 06)

56. Diferencie autenticação, sessão e JWT.

Autenticação confirma quem é o usuário (evento). Sessão mantém o estado autenticado entre requisições (estado). JWT é um formato compacto de token que transporta claims (credencial).

57. Por que enviar `?userId=1` não basta?

Identificador não comprova identidade; qualquer um pode enviar. A API precisa verificar credenciais/token.

58. Por que não reenviar usuário e senha em toda requisição?

Expõe a senha repetidamente. Após o login guarda-se o token (resultado) e ele acompanha as próximas requisições.

59. Quais as três partes de um JWT?

header (algoritmo e tipo) . payload (claims) . signature (integridade).

60. O conteúdo do JWT é secreto? Ler o token prova que ele é válido?

Não é secreto: header e payload são Base64URL (codificados, não criptografados). Não prova: a API precisa verificar assinatura e validade. Nunca coloque senhas no payload.

61. O que é "bearer token" e quais cuidados exige?

Credencial ao portador: quem tem, usa. Proteger o token, usar HTTPS, não registrar em logs.

62. Como enviar o token numa consulta protegida?

`fetch(url, { headers: { Authorization: `Bearer ${accessToken}` } })`. Sem token ou com token inválido a API recusa.

63. Login é `useQuery` ou `useMutation`? E buscar perfil?

Login: `useMutation` (operação imperativa disparada pelo usuário com credenciais). Perfil: `useQuery` (dado consultável). Sair: limpar a sessão.

64. O que guardar na sessão após o login?

`{ userId, accessToken }` — nunca a senha.

65. Para que serve `enabled: sessao !== null` no `usePerfilQuery`? E o token precisa estar na queryKey?

Impede executar a consulta sem sessão. Não: a chave usa `["perfil", sessao?.userId]`.

66. Autenticação × autorização?

Autenticação: "quem é você?". Autorização: "o que você pode fazer?".

Firebase Auth e rotas protegidas (aula 07)

67. Quais responsabilidades o Firebase Authentication assume?

Criar usuários, autenticar credenciais, manter a sessão e informar mudanças no estado de autenticação.

68. Passos no Console para usar email/senha?

Criar projeto → registrar app Web (gera firebaseConfig) → Authentication → Sign-in method → habilitar Email/Password.

69. Diferença entre `createUserWithEmailAndPassword` e `signInWithEmailAndPassword`?

O primeiro cria conta e já autentica; o segundo entra com conta existente.

70. Para que serve `onAuthStateChanged` e por que existe `isLoading`?

Observa o estado de autenticação (recebe User ou null) e o app reage. Ao abrir o app, o Firebase ainda verifica a sessão; `isLoading` evita decidir rotas antes disso.

71. O que o `AuthProvider` expõe?

`user`, `isLoading`, `signUp()`, `signIn()`, `signOut()`.

72. Escreva as rotas: `index` e `event/[id]` públicas; `sign-in/sign-up` só sem usuário; `registrations` só com usuário.

```
<Stack> <Stack.Screen name="index"/> <Stack.Screen name="event/[id]"/> <Stack.Protected guard={!user}>sign-up, sign-in</Stack.Protected> <Stack.Protected guard={!user}>registrations</Stack.Protected> </Stack>
```

73. O que acontece no `logout`?

`signOut(auth)` → `onAuthStateChanged` recebe null → `AuthProvider` zera `user` → `registrations` volta a ficar protegida.

74. Responsabilidade de `lib/firebase.ts`, `providers/AuthProvider.tsx`, `app/_layout.tsx`?

Configurar o Firebase; manter o estado de autenticação; proteger as rotas.

Firestore — BaaS e leitura (aula 08)

75. O que é BaaS? O backend deixa de existir?

Backend as a Service: plataforma que oferece capacidades de backend (identidade, banco, arquivos, funções, notificações) via SDKs/APIs. O backend continua existindo; muda quem constrói e opera (o provedor).

76. Capacidade específica × recorrente (commodity)?

Específica: diferencial do produto (regras de domínio, cálculos exclusivos). Recorrente: aparece em muitos sistemas e tem soluções prontas (login, persistência, arquivos, notificações). Commodity = comum e disponível, não irrelevante.

77. Cite limites/compromissos de adotar BaaS.

Dependência da plataforma, consultas condicionadas ao modelo e índices, cobrança por uso, migração difícil, necessidade de configurar regras de acesso.

78. BaaS e API própria podem coexistir?

Sim, capacidade por capacidade: BaaS para auth/armazenamento/dados diretos; API própria para regras exclusivas, integrações privadas e operações privilegiadas.

79. Como o Firestore organiza dados? Compare com relacional.

Coleções → documentos (com ID) → campos; caminho `posts/7qA2fLm9` identifica o documento. No relacional o banco aplica o esquema; no Firestore a aplicação controla a consistência (documentos podem ter campos diferentes).

80. Escreva a consulta "10 posts mais recentes".

```
query(collection(db, "posts"), orderBy("createdAt", "desc"), limit(10)) e executar com await getDocs(q).
```

81. `collection()` e `query()` executam leitura?

Não. São referência e descrição; só `getDocs` (ou `onSnapshot`) executa.

82. O que contém um `QuerySnapshot`?

`empty` (nenhum doc?), `size` (quantidade), `docs` (lista de `QueryDocumentSnapshot`).

83. Como montar o objeto com id a partir do snapshot?

```
snapshot.docs.map((d) => ({ ...d.data(), id: d.id })) — o id não está em data().
```

84. Por que `d.data()` as Post não basta? Qual a solução?

A conversão só informa o compilador, não valida em tempo de execução. Usar Zod: `postSchema.parse({...d.data(), id: d.id})` — documento inválido vira falha tratável.

85. Firestore query × TanStack useQuery × Zod × RN?

Selecionar dados remotos · controlar execução, cache e estados · validar documentos · apresentar.

86. `getDocs` retornou 10 docs às 10:00; outro cliente criou um às 10:01. O que o snapshot mostra?

Os mesmos 10 — é uma fotografia. Só uma nova execução (`refresh`) gera outro snapshot.

Firestore — escrita, Rules, tempo real, update/delete (aulas 09–11)

87. Qual o contrato de `addDoc`?

`addDoc(CollectionReference, dados): Promise<DocumentReference>` — cria documento com ID automático.

88. O ID automático indica ordem de criação?

Não. Distingue documentos; para ordenar é preciso um campo como `createdAt`.

89. Por que usar `serverTimestamp()` e não `new Date()`?

O relógio do dispositivo pode estar errado ou ser alterado e dispositivos discordam. `serverTimestamp` envia um `FieldValue` especial e o servidor define o `Timestamp`.

90. O que a `DocumentReference` retornada contém e o que não contém?

Contém `id` e `path`. Não contém os campos, o `createdAt` final, um `DocumentSnapshot`, a coleção atualizada nem o cache do TanStack atualizado.

91. Quando a Promise do addDoc resolve/rejeita/fica pendente?

Resolve quando o backend confirma; rejeita quando o backend informa falha (ex.: Rules negaram); fica pendente se o backend não pode ser alcançado.

92. O que é compensação de latência?

O SDK aplica a escrita no estado local antes da confirmação remota — o que aparece localmente não é a confirmação do servidor.

93. Por que "o cliente não é uma fronteira confiável"? Quem autoriza?

Um cliente modificado pode ignorar validação, alterar campos, trocar authorId ou chamar o SDK fora do fluxo. Quem autoriza são as Security Rules, avaliadas no serviço antes da persistência.

94. Escreva a regra: criar post só logado, com campos exatos, content 1–280, autor = logado e horário do servidor.

```
allow create: if request.auth != null && request.resource.data.keys().hasAll([...]) &&
request.resource.data.keys().hasOnly([...]) && request.resource.data.content is string &&
request.resource.data.content.size() > 0 && request.resource.data.content.size() <= 280 && request.resource.data.authorId
== request.auth.uid && request.resource.data.createdAt == request.time;
```

95. Diferença entre allow create e allow write?

write inclui create, update e delete; create autoriza apenas novos documentos.

96. Formulário, TypeScript e Security Rules: papéis?

Feedback antes do envio · coerência de tipos no desenvolvimento · autorização e proteção do dado no servidor. "Validação no cliente melhora o uso; regra no backend protege o dado."

97. A mutation precisa usar HTTP?

Não. A mutationFn só precisa receber variáveis e devolver uma Promise (ex.: addDoc).

98. Depois do addDoc, por que invalidar ["posts"] se o Firestore já tem o doc?

Há dois caches: o do SDK Firestore e o do TanStack Query. A escrita não reescreve o resultado da leitura anterior; invalidar gera novo getDocs.

99. Por que não ativar retry na mutation de addDoc?

Cada execução gera um novo ID → documentos duplicados se a primeira tiver sido concluída.

100. Diferencie getDocs e onSnapshot.

getDocs: leitura pontual, retorna Promise<QuerySnapshot> e termina. onSnapshot: assinatura; entrega o estado inicial e novos snapshots a cada mudança via callback; retorna Unsubscribe.

101. Preciso chamar getDocs antes do onSnapshot?

Não: a primeira emissão já traz o estado atual da consulta.

102. O que acontece se o listener receber erro?

O callback de erro (3º argumento) recebe a falha terminal (ex.: permissão insuficiente, consulta inválida) e o listener para de emitir.

103. Por que usar useFocusEffect em vez de useEffect para o listener?

Em navegação em pilha a tela continua montada quando outra fica visível; o cleanup do useEffect só ocorreria no unmount, mantendo o listener ativo. useFocusEffect assina no foco e cancela ao perder o foco.

104. Escreva o listener no foco da tela.

```
useFocusEffect(useCallback(() => onSnapshot(q, (s) => setPosts(s.docs.map(mapPost)), (e) => setError(e)), [])); — o
retorno (unsubscribe) é o cleanup.
```

105. Quais as 4 etapas de uma assinatura?

Começo, emissões, falha e encerramento.

106. Por que título ou posição na lista não servem para identificar o documento a editar?

Título pode repetir/mudar; posição muda com ordenação, inserção ou remoção. Use doc.id preservado no item.

107. Escreva completeTask(taskId) e removeTask(taskId).

```
await updateDoc(doc(db, "tasks", taskId), { completed: true }); await deleteDoc(doc(db, "tasks", taskId)).
```

108. updateDoc com { completed: true } apaga title e subject?

Não: atualização parcial altera só os campos informados e preserva o resto. Falha se o documento não existir.

109. O card sumiu da tela. O documento foi excluído?

Não necessariamente: filtrar o estado local ou escrita local pendente também faz sumir. A confirmação é a Promise do deleteDoc resolver (e conferir no Console).

110. Esconder o botão "excluir" protege o dado?

Não. Identificar o caminho não concede permissão; quem decide são as Security Rules.

111. Por que pedir confirmação antes do deleteDoc?

É ação destrutiva e irreversível (não há lixeira/restauração) — também é recomendação de acessibilidade/design.

112. Com listener ativo, a lista atualiza antes ou depois da Promise da mutation?

Pode atualizar antes (o SDK aplica a mudança local); a mutation continua isPending até o servidor confirmar. Com leitura pontual, é preciso invalidar após o sucesso.

113. Qual API do Firestore para cada operação do CRUD?

Create: addDoc (coleção). Read: getDocs/onSnapshot. Update: updateDoc (documento). Delete: deleteDoc (documento).

Dissertativas curtas (modelo de resposta)

114. Explique o fluxo completo de criar um post com Firestore + TanStack Query.

O usuário toca em publicar → a tela chama `mutate(input)` do `useCreatePostMutation` → a `mutationFn` `createPost` chama `addDoc(postsCollection, {...input, createdAt: serverTimestamp()})` → as Security Rules avaliam (auth, campos, autor, horário) → se negarem, a Promise rejeita e a UI mostra erro (`isError`); se permitirem, o doc é criado e a Promise resolve com a `DocumentReference` → `onSuccess` invalida `["posts"]` → a query ativa faz novo `getDocs` e a lista atualiza. Durante tudo isso, `isPending` desabilita o botão.

115. Quando escolher `getDocs` + `useQuery` e quando `onSnapshot`?

`getDocs+useQuery` quando basta uma fotografia atualizada em oportunidades (abrir tela, puxar para atualizar), aproveitando `cache/retry/estados` do TanStack. `onSnapshot` quando a tela precisa refletir mudanças de outros clientes em tempo real (mural, chat), assumindo o custo de manter o listener — e encerrando-o ao perder o foco.

116. Por que separar `services`, `hooks` e telas?

Separação de responsabilidades (coesão), menos acoplamento da UI com a biblioteca/API, configuração centralizada (`queryKey/retry/invalidação` iguais em todas as telas), facilidade para trocar o transporte (`fetch` → `Firestore`) sem mexer na tela, e telas que leem como intenções.